FOUNDATIONS OF WEB APP PENTESTING

1.2 CLIENT-SERVER MODEL



Client-Server Architecture • Request-Response Cycle • Sessions & Cookies

HAROON SHAHID

FOUNDATIONS OF WEB APP PENTESTING

1.2 CLIENT-SERVER MODEL

CLIENT–SERVER ARCHITECTURE

What is this?

Every web application works using **two main parts**:

- Client → asks for something
- Server → gives the answer

I. THE CLIENT (Requester)

Simple meaning:

The client is the thing that sends requests.

In real life:

- Your browser (Chrome, Firefox, Brave)
- Or tools like curl

What it does:

- You type a website
- You click buttons
- You submit forms

All of this creates a **request**

II. THE SERVER (Provider)

Simple meaning:

The server is a computer that gives responses.

What it stores:

- Website files (HTML, CSS, JS)
- Data (users, passwords, posts)

- Application logic

What it does:

- Waits for requests
- Processes them
- Sends data back

SIMPLE REAL-LIFE EXAMPLE

Think of a restaurant:

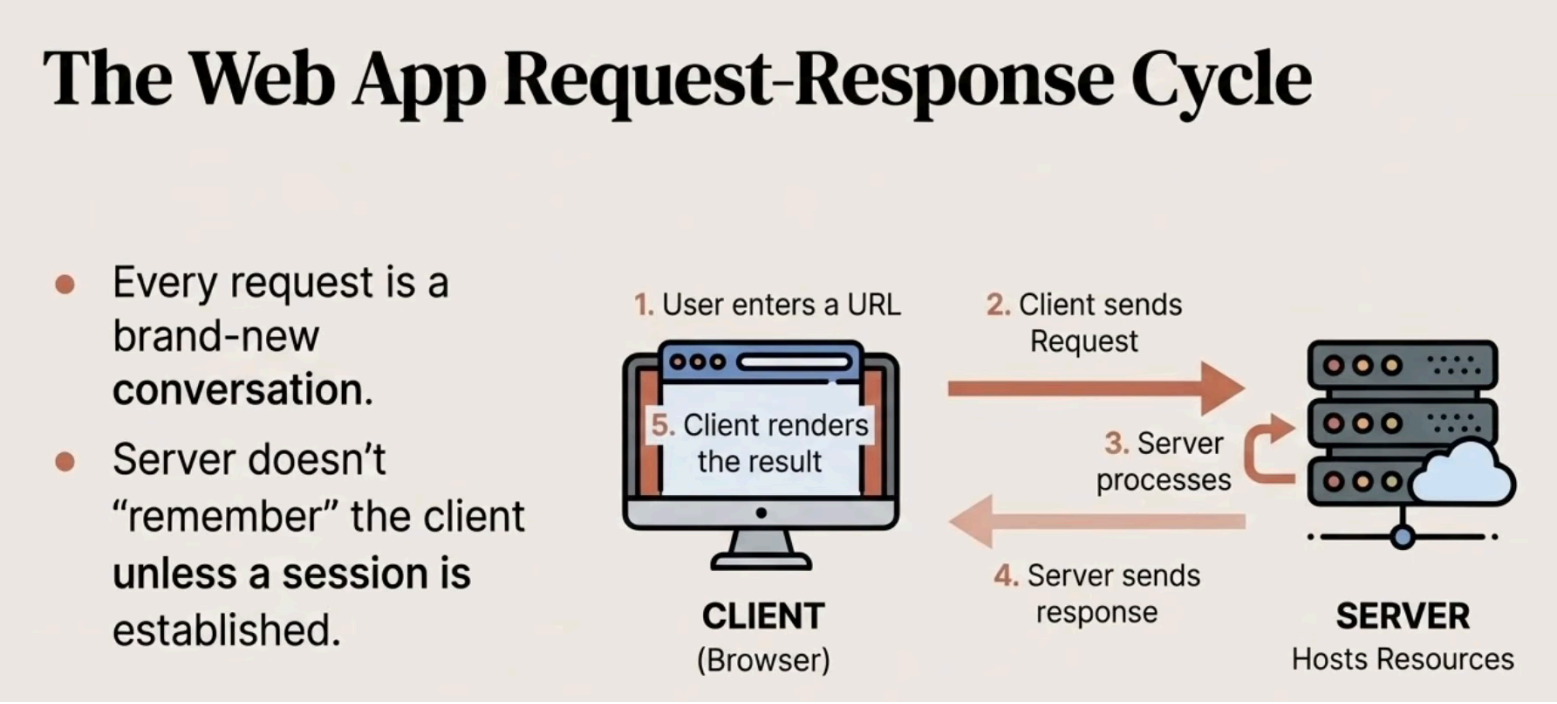
- You (client) → order food
- Kitchen (server) → prepares food

You don't go into the kitchen.
You just request → and get a response.

THE REQUEST–RESPONSE CYCLE (CORE FLOW)

This is the **most important concept** in web pentesting.

Every website works using this exact flow.



STEP 1 — URL ENTRY

You type a website in the browser.

Example:

```
example.com
```

This tells the browser:

“Go get this website”

STEP 2 — THE REQUEST

The browser sends a request to the server.

Example (simple idea):

“Give me the homepage file”

In reality, it looks like this:

```
GET /index.html
```

This is called an **HTTP Request**

STEP 3 — PROCESSING (SERVER SIDE)

The server receives the request and decides:

- Does the file exist?
- Does user have permission?
- Do I need to check database?

Example:

- Login request → check username/password
- Search request → fetch data from database

STEP 4 — THE RESPONSE

The server sends back data.

This includes:

- HTML (structure)
- CSS (design)
- JavaScript (behavior)
- Status code (important)

Example status codes:

- 200 → OK
- 404 → Not Found
- 500 → Server Error

This is called an **HTTP Response**

STEP 5 — RENDERING (BROWSER)

The browser receives raw code and converts it into a visible page.

Example:

- HTML → structure
- CSS → design
- JS → interaction

You now see a working website

SIMPLE FLOW SUMMARY

User → Request → Server → Response → Browser shows result

WEB PROXY (YOUR MAIN WEAPON)

Now this is where pentesting actually starts.

Normally: `Client → Server`

But in pentesting: `Client → Proxy → Server`

WHAT IS A PROXY?

A proxy is a tool that sits **between you and the server**

COMMON TOOLS

- Burp Suite
- Caido

WHAT IT DOES

1. INTERCEPTING (STOP REQUEST)

You can catch a request before it reaches the server.

Example:

```
POST /login
username=haroon
password=123456
```

You can stop it and inspect it.

2. MANIPULATION (CHANGE DATA)

You can modify the request.

Example:

- Change username
- Change price
- Change role

Example attack idea:

```
price=100 → change to → price=1
```

Then send it to server.

3. INTERCEPT RESPONSE

You can also stop server response before browser sees it.

Example:

- Change “Access Denied” → “Access Granted”
- Modify returned data

REALITY CHECK

Most pentesting work = Watching and modifying requests in proxy

If you don’t master this → you’re not really pentesting

STATELESSNESS & SESSION MANAGEMENT

This is where most beginners get confused.

STATELESSNESS (VERY IMPORTANT)

Simple meaning:

The server does NOT remember you.

Every request = new request

EXAMPLE

You login to a website.

Then you click:

- Profile
- Settings
- Dashboard

Each click = separate request

Server does NOT automatically remember:
“this is Haroon”

SO HOW DOES IT REMEMBER YOU?

This is where **sessions** come in.

SESSION (IDENTITY TRACKING)

A session is a way to remember a user.

HOW IT WORKS

After login, server gives you an ID.

Example:

```
session_id = abc123
```

This is stored in your browser.

IDENTIFIERS USED

1. Cookies

Small data stored in browser.

Example:

```
Cookie: session_id=abc123
```

Browser sends this in every request.

Cookies are like the vehicles that carry the `session_id` to the server on every request. And the `session_id` is a **string of characters and numbers** that the server gives to the client in return for its **credentials**.

2. JWT (JSON Web Token)

A special token that proves your identity.

Used in modern apps.

SIMPLE IDEA

Without session:

- Server forgets you every time

With session:

- Server knows who you are

PENTESTER FOCUS (VERY IMPORTANT)

Most real attacks happen here.

WHAT ATTACKERS TRY

- Steal cookies
- Predict tokens
- Hijack another user session

EXAMPLE ATTACK (SESSION HIJACKING)

1. Victim logs in
2. Attacker steals cookie
3. Attacker uses same cookie

Server thinks attacker = victim

FINAL SUMMARY (CORE UNDERSTANDING)

Client:

- Sends request

Server:

- Processes request
- Sends response

Request–Response:

- The full communication cycle

Proxy:

- Lets you see and modify traffic

Stateless:

- Server does not remember you

Session:

- Makes server remember you

SESSION, SESSION ID, COOKIES & SESSION COOKIES

When you log into a website, the server needs a way to **remember you**.

It does NOT remember you automatically.

So it uses:

- Session
- Session ID
- Cookie

These work **together**, not separately.

1. SESSION (THE MEMORY ON SERVER)

Simple meaning:

A session = your login state stored on the server

It's like:

"This user is logged in"

Example:

You log into a website.

Server creates:

```
Session for Haroon:
- user_id = 101
- logged_in = true
```

This is stored **on the server**, not in your browser.

Important:

- Session = **data on server**
- You cannot see it directly

2. SESSION ID (THE IDENTIFIER)

Simple meaning:

A **session ID = unique ID that represents your session**

It's like a **ticket number**

Example:

Server creates: `session_id = abc123`

This ID points to your session on the server.

Real-life analogy:

- Session = your hotel room
- Session ID = your room number

Important:

- Session ID is **very sensitive**
- If attacker gets it → they become you

3. COOKIE (STORAGE IN BROWSER)

Simple meaning:

A **cookie = small piece of data stored in your browser**

What it stores:

Usually: `session_id = abc123`

Example:

After login, server sends: `Set-Cookie: session_id=abc123`

Browser stores it.

Now every request sends: `Cookie: session_id=abc123`

Important:

- Cookie lives in **browser (client side)**
- It is just storage, not logic

4. SESSION COOKIE (TYPE OF COOKIE)

Simple meaning:

A **session cookie = a cookie that stores session ID temporarily**

Key point:

- It gets deleted when browser closes

Example:

You login → session cookie created
You close browser → session gone

Compare:

Type	Behavior

Session Cookie	deleted on browser close
Persistent Cookie	stays even after restart

NOW CONNECT EVERYTHING

Step-by-step flow:

1. You login
2. Server creates:
    - Session (data)
    - Session ID (abc123)
3. Server sends:

```
Set-Cookie: session_id=abc123
```

4. Browser stores it (cookie)
5. Every request sends:

```
Cookie: session_id=abc123
```

6. Server checks:

- “Oh, this is session abc123”
- “This user is Haroon”

That’s how you stay logged in

MOST COMMON CONFUSION

Session = Cookie

WRONG

Session → server memory
Cookie → browser storage

Session ID = Session

WRONG

Session → actual data
Session ID → pointer to that data

Cookie = always session

WRONG

Cookie can store anything
Session cookie specifically stores session ID

SUPER SIMPLE ONE-LINE DEFINITIONS

- Session → user data stored on server
- Session ID → unique ID of that session
- Cookie → small storage in browser
- Session Cookie → cookie that stores session ID temporarily

REAL ATTACK UNDERSTANDING (IMPORTANT)

If attacker steals: `session_id = abc123`

They can send: `Cookie: session_id=abc123`

Server thinks: “This is Haroon”

Attacker is now logged in as you

This is called:

- Session hijacking
